Compte rendu : TP 1

Réalisé par LOARI Moumouni Boukary

Chiffrement authentifié et dérivation de clés (coffre.py)

1. Objectif et démarche

Réaliser un mini-coffre de fichiers sans réimplémenter aucune primitive : **Argon2id** dérive une clé maîtresse du mot de passe, **HKDF-SHA256** en tire une sous-clé par usage (`info = b"coffre-v1|" + usage`), **AES-256-GCM** chiffre et authentifie. Le fichier produit est : `CFR1` | t, m, p (3 × 4 octets) | sel (16) | nonce (12) | chiffré + étiquette (16). L'en-tête de 44 octets, suivi du nom du fichier, forme les **données associées** (AAD) : il est authentifié sans être chiffré.

2. Réglage du coût (étape 1)

`python coffre.py` chronomètre quatre jeux de paramètres avec `mesurer_parametres` ; le jeu retenu s'appelle par `mesurer_parametres()` sans argument (il prend `T, M, P`). Les durées varient d'une exécution à l'autre (la première inclut l'initialisation) ; on répète donc la mesure.

Paramètres (t, m, p)	Durée mesurée	Commentaire
1, 32 Mio, 4	0,026 s	Trop faible
2, 64 Mio, 4	0,077 s	Trop faible
3, 64 Mio, 4	0,104 s	Insuffisant
3, 128 Mio, 4	0,222 s	Correct mais en dessous de la cible
4, 256 Mio, 4 (retenu)	**0,49 s ≈ 0,5 s**	Deux fois plus de mémoire, plus de passes : proche de la cible

Choix : environ 0,5 s par dérivation, supportable pour l'utilisateur et coûteux pour l'attaquant. On privilégie la **mémoire** (m), qui limite les attaques par GPU, puis t ; p suit le nombre de cœurs.

3. Format, tests et évolution (étapes 2 à 4)

`chiffrer` tire un sel et un nonce neufs avec `os.urandom` ; `dechiffrer` relit t, m, p dans l'en-tête et refuse un format inconnu (`FormatInconnu`). La suite `pytest` compte **8 tests, tous réussis** (`8 passed`) :

Test	Résultat	Propriété
aller-retour ; chiffrement probabiliste	Données retrouvées ; deux sorties différentes	Correction, sel et nonce neufs
mauvais mot de passe ; octet modifié	InvalidTag	Authentification de la clé et intégrité
en-tête modifié ; nom échangé	InvalidTag	L'en-tête et le nom sont authentifiés (AAD)
format inconnu	FormatInconnu	Refus d'une version inconnue
rechiffrer / évolution	Réussi	Ancien fichier lisible après durcissement de T et M

Comme t, m, p sont écrits dans chaque fichier, on peut durcir les constantes sans rendre illisibles les anciens fichiers ; rechiffrer migre un fichier vers les paramètres courants et sert aussi à changer le mot de passe. Aucune clé ni mot de passe n'est conservé dans les livrables.

4. Questions de réflexion

1. **Sel en clair ?** Il n'a pas à être secret : il rend la dérivation unique par fichier, donc inutilise les tables précalculées et force un calcul Argon2id par fichier et par essai. Le publier n'affaiblit rien.
2. **Pas de répétition du nonce ?** Le danger de GCM est de réutiliser (clé, nonce). La clé dépend d'un sel aléatoire de 128 bits propre au fichier : chaque clé ne chiffre qu'un seul message.
3. **Nom absent des AAD ?** Un attaquant pourrait échanger deux blobs (ex. virement_autorise.dat et brouillon.dat) : chacun reste intact, l'étiquette est valide, et le programme lirait le mauvais contenu. Avec le nom dans les AAD, l'échange fait échouer l'étiquette (test_nom_echange).
4. **Authentifier l'en-tête ?** Il contient t, m, p et le sel, lus avant le déchiffrement. Modifiable, il permettrait d'imposer une dérivation faible ou d'épuiser la mémoire (déni de service). Un contrôle de plage sur t, m, p reste conseillé.
5. **Cacher la raison d'un échec ?** Distinguer mauvais mot de passe, fichier corrompu et mauvais nom donnerait un oracle à l'attaquant. AES-GCM produit d'ailleurs le même InvalidTag ; le message reste unique (« déchiffrement impossible »), le détail va dans un journal interne.